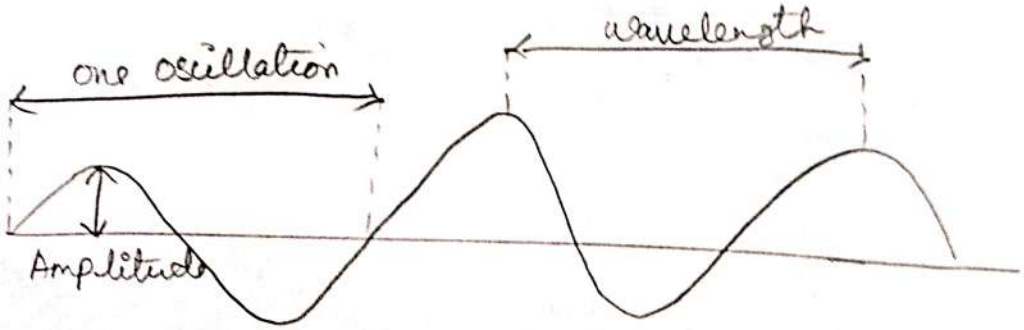
Pic microcontroller

1) Active Components	Passive components
* It requires Power Source	* It do not need Power Source
* It amplify, direct and control the component	* It limit, dissipate or store the energy
* Transistor, Diode, LED Photodiode, IC, operational Amplifier Seven segment Display, Battery	* Resistor, LDR, Thermistor, Capacitor Inductor Switch variable resistor transformer

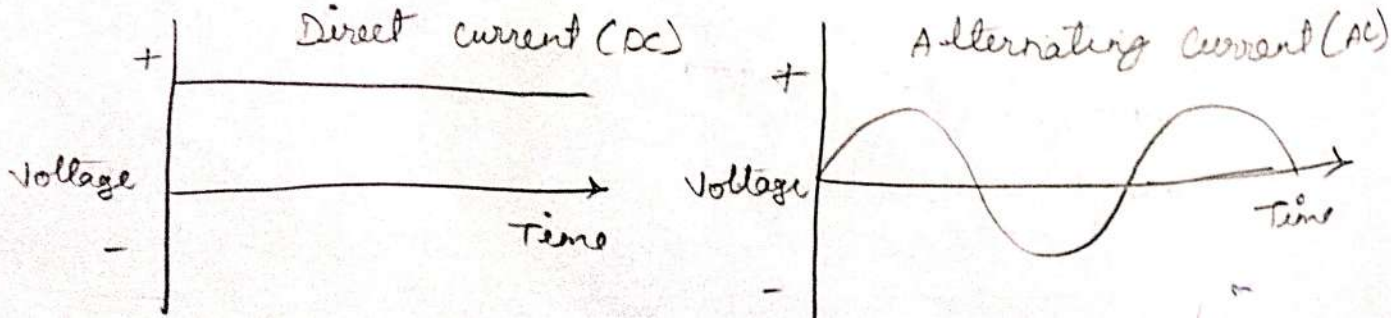
Frequency

$$f = \frac{1}{T}$$

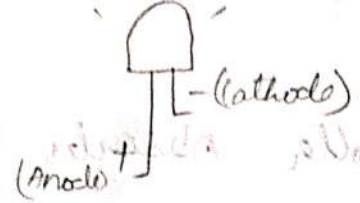


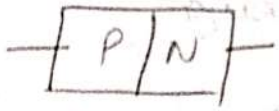
Frequency - no. of oscillation Per Sec

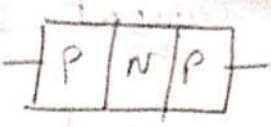
AC & DC

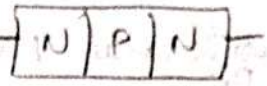


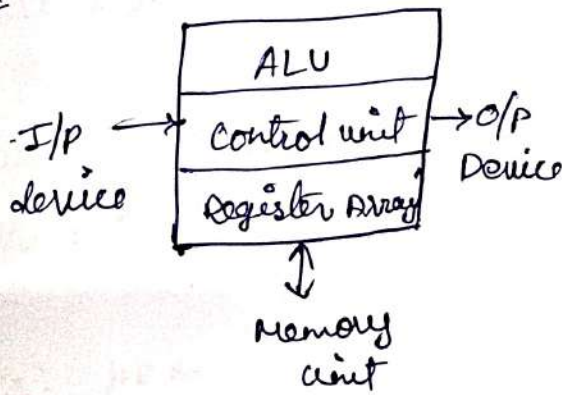
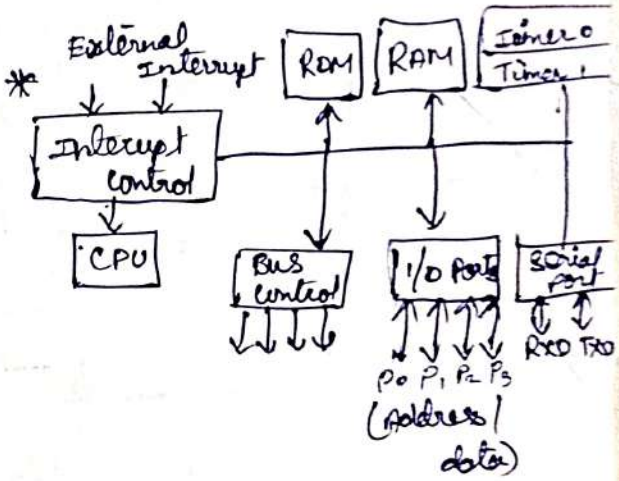
Anode → +Ve
Cathode → -Ve





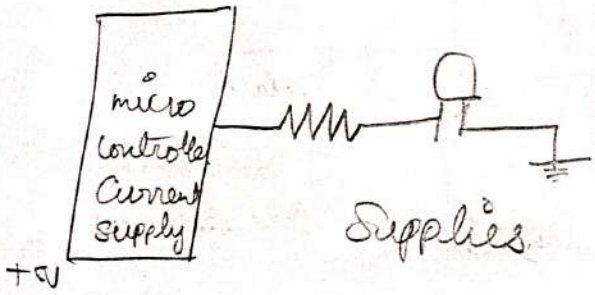




Microprocessor	Microcontroller
* It can performs a General tasks	* It performs a specific task
* 	* 

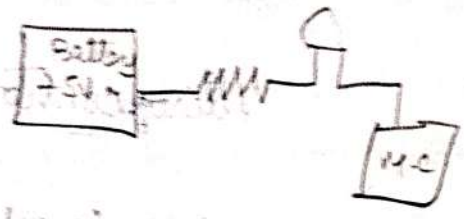
Current Sourcing

* Current Source is where the current is comes from. it supply the current

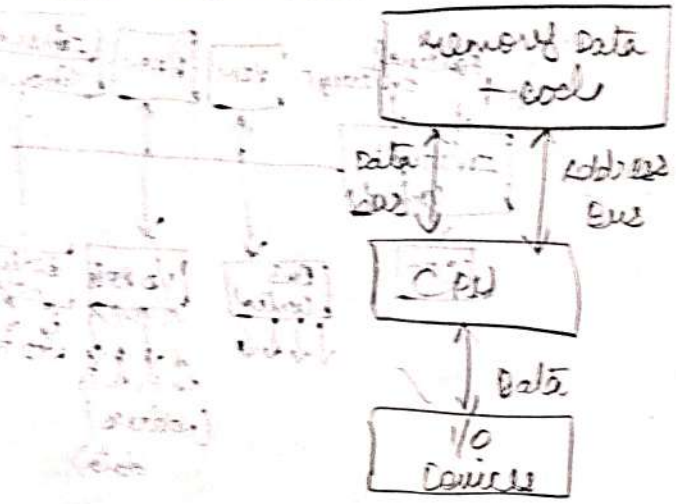


Current sinking

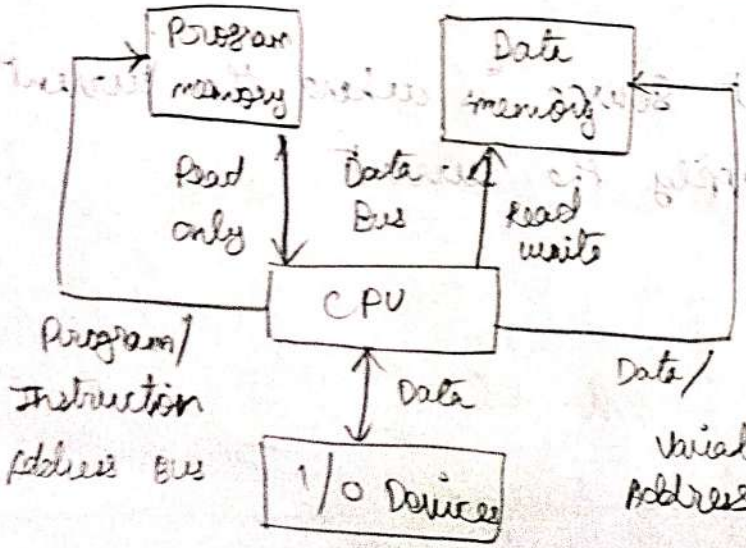
* Here the microcontroller absorbs the current from external power supply. That's why we more PIC preferred ~~the current sinking~~ contains current sink



Von neumann



Harvard



RISC	CISC
* Feature → RISC (Reduced instruction set computer)	* CISC (Complex Instruction Set Computer)
* Focus → Software: compliers do more work	* Hardware: Transistors loads complex logic
* Instruction → simple, single cycle executions	* Complex, multi-cycle executions
* Code size Large	* Code size smaller
* Power use highly efficient, low Power.	* power-hungry, higher heat
* Ex: ARM	* Intel x86

PIC (PIC16F877A) → current version

* A popular 40 Pin, 8-bit CMOS microcontroller by microchip

Core Specification:

*. CPU: 8-bit RISC architecture (10-bit ADC)
* Operating Freq: DC to 20 MHZ
* I/O Pins: 33 Programmable pins across 5ports (A, B, C, D, E)
* Memory: 14KB Flash, 368 Bytes RAM, 256 Bytes EEPROM.

RISC	CISC
* Feature → RISC (Reduced instruction set computer)	* CISC (Complex Instruction Set Computer)
* Focus → Software: compliere do more work	* Hardware: Transistors used complex logic
* Instruction → simple, single cycle executions	* Complex, multi-cycle executions
* Code size Large	* Code size smaller
* Power use highly efficient, low power.	* Power-hungry, higher heat
* Ex: ARM	* Intel x86

PIC (PIC16F877A) → current version

* A popular 40 Pin, 8-bit CMOS microcontroller by microchip

Core specification:

* CPU: 8-bit RISC architecture (10-bit ADC)
* Operating Freq: DC to 20 MHZ
* I/O Pins: 33 Programmable pins across 5 ports (A, B, C, D, E)
* Memory: 14KB Flash, 368 Bytes RAM, 256 Bytes EEPROM. (Password)

Total 40 Pins in PIC16F877A but for I/O Pins are 33 Pins.

Port	Pins	Specification
A →	RA0-RA5 (6 Pins) →	Analog I/O
B →	RB0-RB7 (8 Pins) →	Interrupt
C →	RC0-RC7 (8 Pins) →	Communication Protocol
D →	RD0-RD8 (8 Pins) →	Data display
E →	3 Pins →	Extra

Configure

* Here, If we convert this Ports to Program Code we use TRIS

Ex: Port A → TRISA in Code
Port B → TRISB

TRISB = 0x00 → 0000 | 0000
0xAF → 1001 | 1111

In TRIS we take 0's / 1's as the o/p & i/p

0 → o/p
1 → I/p

TRIS is the direction of Ports it has 3 state

- 1 → High (ON)
- 0 → Low (OFF)
- High-impedance (DIM)

} Tri State buffer

In ports we take 0's / 1's as low & high

0 → Low

1 → high

MPLAB X IDE v5.40 → download for program | XC8-V3.10 | Proteus 8 Professional

8-bit

Step 1 → File → New Project → Click Next below → In device Base PIC16F877A → next → click XC8-compiler → Project name → next → Left side Source file → new → main.C → file name type → Finish → Production TOP → set configuration → below we can change it → Generate source code o/p below click → copy configuration → and paste header top or bottom → set frequency #define _XTAL_FREQ 2000000 → Inside main() write Port TRIS state → Left side Program name Right click → build

Ex: #define _XTAL_FREQ 2000000

```
void main (void){
    TRISB = 0x00;
    while(1){
        PORTBbits.RB0 = 1;
    }
}
```

Proteus Design open → File → New project → Name → next → next → next → next → then the interface is created → left side white space double click → Keywords Above type PIC16F877A → OK → to click center → 1 → type LED → In LED we can choose colours → OK → After fixing Components → double on chip → Processor clock Freq change 20MHz → Click file image → local disk C → users → 'Some Name → MBLAB

Ex: void main(void){

```
TRISB = 0x00;
while(1){
  RB0 = 1;
  __delay_ms(500);
  RB0 = 0;
  __delay_ms(500);
}
return 0;
}
```

PORTBbits.RB0 = !PORTBbits.RB0;

↓

It toggle the LED if the in ON state it change the state into OFF

Proteus Design open → file - New project → name → next → next → next → next → then the interface is created → left side white space double click → Keywords above type PIC16F877A → OK → to click center → ✓ → type LED → In LED we can choose colours → OK → After typing Components → double on chip → Processor clock Freq change 20MHz → Click file image → local disk C → users → 'Some Name → MBLAB'

Ex: void main(void){

```
TRISB = 0x00;
while(1){
  RB0 = 1;
  __delay_ms(300);
  RB0 = 0;
  __delay_ms(500);
}
return 0;
}
```

PORTBbits.RB0 = !PORTBbits.RB0;

↓

It toggle the LED if the in ON state it change the state into OFF

Microcontroller Timer

* In embedded s/m, time mangement is critical
* A micrcon needs to know exactly how long an event took, or it needs to trigger an action at a precise interval
* A timer is a dedicated piece of h/w built inside the micrcon whose sole purpose is to keep track of time or count events.
* It operates independently from the main CPU, meaning it can count in the background while the CPU executes your C program.

The Real-world Analogy

* Think of a micrcon without a timer like a person working a pizza who has to stare at the oven the entire time so it doesn't burn.
* They can't chop vegetables or wash dishes; their full attention is on the time. This is what s/w delays do to a micrcon.
* A h/w timer is like a kitchen timer. You set it for 15 min and then you are free to do other tasks. When the time is up, the timer rings an alarm to get your attention back.

Why Do we need Hw Timers?

* The standard way to pause a program is using a delay function. This is called "blocking code" because it blocks the microcontroller from doing anything else.

* If you use a 1-sec Sw delay, the microcon is completely frozen for the entire sec.

* It cannot read a button press, it cannot update a display, and it cannot communicate with other sensors.

* Hw Timers provide "non-blocking" timing, freeing up 100% of the CPU's processing power.

How a Hw Timer work

* At its core, a h/w timer is simply a digital counter.

* It consists of a special register (memory location) that increment its value by 1 every time it receives a clock pulse.

* These clock pulses usually come from the S/m's main oscillator (the crystal.

* If we know the frequency of the crystal, we

* By counting the no. of pulses. the microcon can precisely measure elapsed time

Timer. watch Dog Timer

watch Dog Time :-

* A watchdog timer is a specialized, fail-safe timer designed to monitor system health, automatically resetting the Processor if the software freezes, crashes or falls into an endless loops.

Timer & counters in Embedded S/m

Timer → measures time intervals
Counter → Counts external events

common Application

* Generating delays
* measuring signal freq or pulse width
* Producing PWM signals
* Counting pulses from rotary encoders
* Implementing basic real-time functions

The concept of timer overflow

* Every timer has a maximum capacity based on its size

* An 8-bit timer can count from 0 to 255, and a 16-bit timer can count from 0 to 65,535.

* what happens when an 8-bit timer reaches 255 & receives one more pulse? it Rolls over back to 0.
* This event is called an "overflow".
* In embedded programming, the overflow is the most important event, because the hardware timer uses this exact moment to signal the main CPU that a specific time period has finished.

Prescaler	Postscaler
* Placed before the timer's counter * Divides the incoming clock freq to increment the timer slower (e.g. 1:2, 1:64) * Extends the max duration a timer can measure before resetting	* Placed after the timer's counter * Counts the no. of times the timer overflows/matches before triggering an interrupt * Reduces the freq of interrupts or watchdog Timer (WDT) resets to free up CPU cycles.

Timers in the PIC16F877A microcontroller

The PIC16F877A comes equipped with three distinct hw timers, giving you incredible flexibility for multitasking

* Timer 0 (TMR0): An 8-bit timer/counter
* Timer 1 (TMR1): A 16-bit timer/counter that can

operate even when the microcon is in sleep mode.

* Timer 2 (TMR2): An 8-bit timer with an internal period register; Primarily used for generating PWM (pulse width modulation) signals for motor control

Interrupts: The H/w Alarm Sys.

* when Timer 0 overflows from 255 to 0, it raises a flag

* But the CPU needs a way to notice this flag. This is where "Interrupts" come in.

* An interrupt is a H/w signal that violently interrupt the main program.

* No matter what the CPU is doing the main() loop an interrupt forces it to immediately pause, jump to a special "Interrupt service Routine" (ISR) to handle the timer event, & then return right back to where it left off.

Calculating the Timer Values

$f = \frac{1}{T} = \frac{1}{0.25 \times 10^{-6}}$

To get a 10ms interrupt with a 16MHz Crystal we use the timer formula.

First, we find the instruction cycle time (Tcy): 16 MHz / 4 = 4MHz, so one cycle is 0.25us.

We will use a 1:256 Prescaler. (it takes the 256 as 1)

TMR0 Register

$1000 = 0.25 \times 256 \times (256 - TMR0_Value)$

$1000 = 64 \times (256 - TMR0_Value)$

$156.25 = 256 - $ "

$TMR0_Value \approx 100$

REGISTER → OPTION_REG REGISTER (ADDRESS 81h, 181h)

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

↓
If T0CS = 0 → It counts the clock pulses → Timer mode
" " = 1 → It counts the event → Counter mode

bit 7 → RBPU : PORTB pull-up Enable bit

bit 6 → INTEDG : Interrupt edge select bit

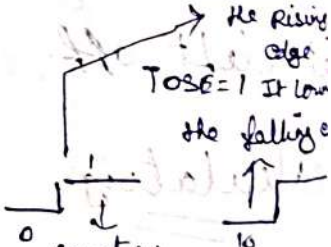
bit 5 → T0CS : TMR0 clock source select bit (It enable the timer 0)

bit 4 → T0SE : TMR0 source Edge " (It enable in Counter mode
If T0SE = 0 It co
the Rising edge
T0SE = 1 It co
the falling e

bit 3 → PSA : Prescaler Assignment bit

bit 2-0 → PS2 : PS0 : " Rate select bits

↳ (It gives the ratio value of the Prescaler in binary)



INTCON REGISTER (Interrupts

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
GIE	PEIE	TMR0IE	INTE	RBIE	TMR0IF	INTF	RBIF
bit 7							bit 0

Types of interrupt

* H/w Interrupt → Triggered by physical electrical signal or internal h/w modules (timers, analog converters)

* S/w Interrupt → Triggered by specific instruction within the S/w itself

* The PIC16F877A relies primarily on H/w Intr for day to day embedded control

Seven Segment Display

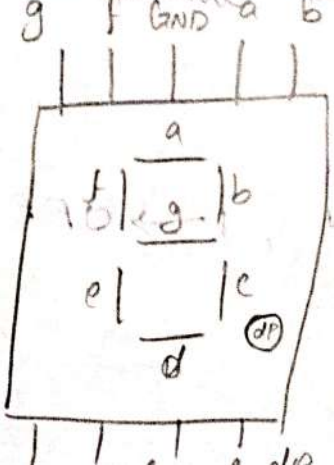
* A seven-segment display is an electronic component used to display decimal numerals (0-9)

* It consists of seven individual Light Emitting Diodes (LEDs) arranged in a figure-8 pattern

* The segment are universally labeled with letters: a, b, c, d, e, f & g

* many displays include an eight LED for the decimal Point (DP)

PIN Diagram



Types of interrupt

* H/w Interrupt → Triggered by physical electrical signal or internal h/w modules (timers, analog converters)

* S/w Interrupt → Triggered by specific instruction within the S/w itself

* The PIC16F877A relies primarily on H/w interrupts for day to day embedded control

Seven Segment Display

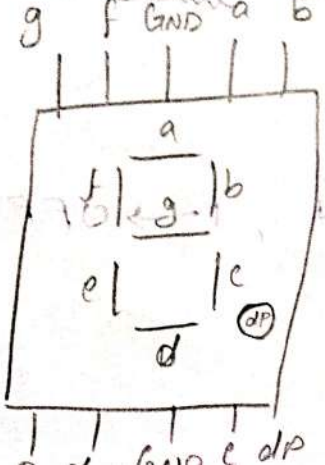
* A seven-segment display is an electronic component used to display decimal numerals (0-9)

* It consists of seven individual Light Emitting Diodes (LEDs) arranged in a figure-8 pattern

* The segment are universally labeled with letters: a, b, c, d, e, f & g

* many displays include an eight LED for the decimal Point (DP)

PIN Diagram



Types :-

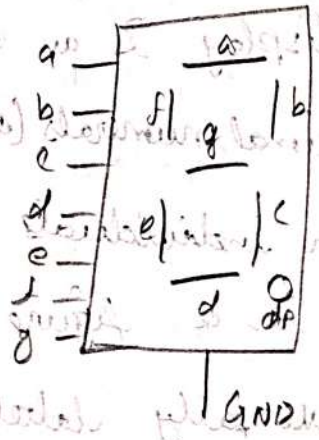
1) Common Cathode (CC)
2) Common Anode (CA)

Common Cathode (CC)

* All cathodes connected to GND
* segment turns ON when pin = HIGH (1)

Logic

1 → ON , 0 → OFF

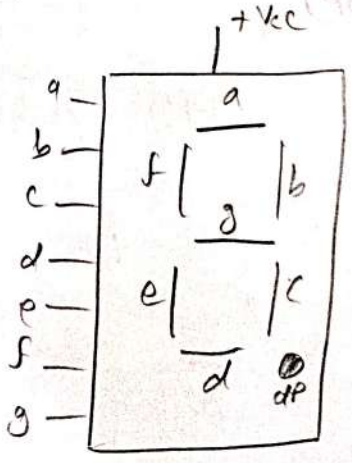


Common Anode (CA)

* All Anode connected to Vcc
* segment turns ON when PIN = LOW (0)

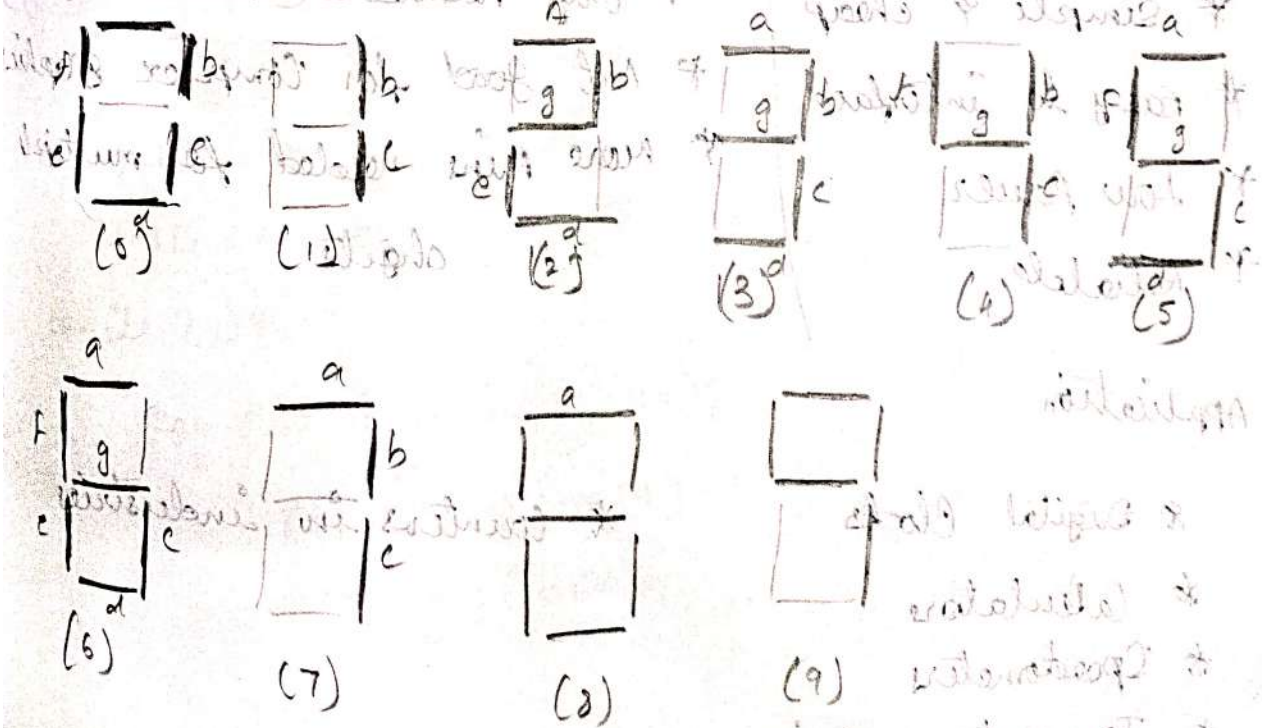
Logic :-

0 → ON , 1 → OFF



Common Cathode	Common Anode
Ex: Arduino digital counter PIC based stopwatch Embedded system training kit student mini-project	Ex: Railway station display Industrial counters Freq meters weighing machines
why CC is Preferred * Direct logic control (1-ON) * Easy to debug & program * compatible with most microcon	* why CA is Preferred * compatible with decoder/driver ICs such as: 7447 BCD-to-7-segment decoder * Better suited for current-sinking drives widely used in commercial electronic

Ex:



Segment Logic Table

For Common Cathode displays

Digit	binary i/p values (F i c)	hexa i/p Value (CC)	CA
0	11111100	FC	03
1	01100000	60	9F
2	11011010	DA	25
3	11110010	F2	0D
4	01100110	66	99
5	10110110	B6	49
6	10111110	BE	41
7	11100000	E0	1F
8	11111110	FE	01
9	11110110	F6	09

Advantages

* Simple & cheap
* Easy to interface
* Low Power
* Reliable

Limitation

* only numbers (limited letters)
* Not good for complex graphics
* more pins needed for multiple digits

Application

* Digital Clocks
* Calculators
* Speedometers
* Temperature displays
* Counters in industries

```
#include <xc.h>

#pragma ...
        ...

#define _XTAL_FREQ 20000000

unsigned char digit[10] =
{
0x3F, //0
0x06, //1
0x5B, //2
0x4F, //3
0x66, //4
0x6D, //5
0x7D, //6
0x07, //7
0x7F, //8
0x6F //9
};

void main(){
unsigned char i;
TRISD = 0x00;
while(1){
for(int i=0; i<10; i++){
PORTD = digit[i];
__delay_ms(1000);
}}
}
```

Analog to Digital Converter (ADC)

The real world vs The digital world

* the real world is Analog: Temperature, Sound, Pressure, and light vary continuously

* microcontrollers are digital: they only understand binary data (0s & 1s)

* The Bridge: An - Analog - to - Digital Converter (ADC) translates continuous physical signals into discrete digit numbers.

what is an ADC?

An electronic component (often built directly into the microcon) that converts an analog voltage i/p into a proportional digital o/p code

i/p: Analog Vtg, Reference Vtg [V_ref] & a Clock Sgnl

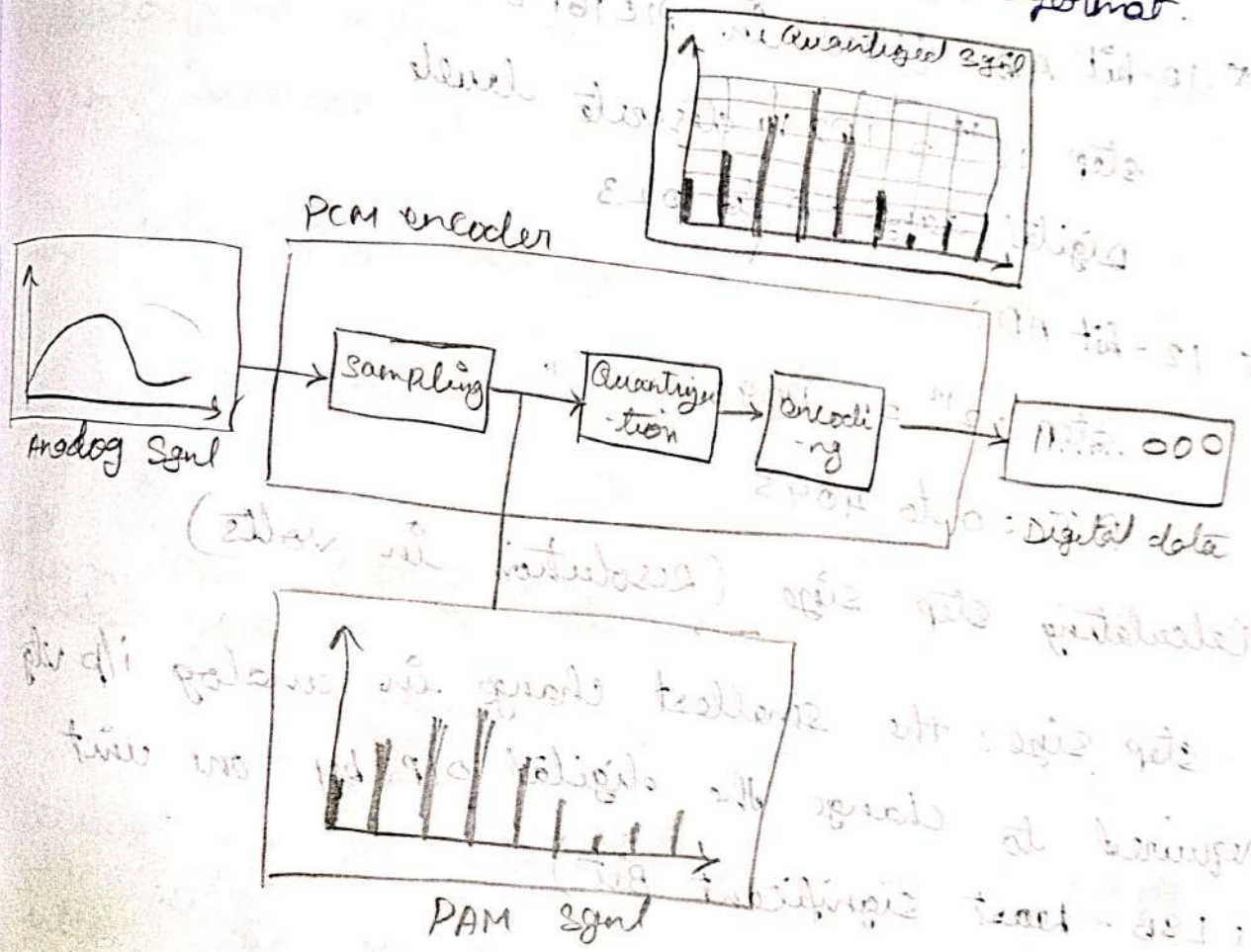
o/p: An n-bit binary Numbers.

The 3-step ADC Process

1) sampling - the ADC takes discrete "snapshots" of the continuous analog sgnl at specific, regular time intervals

2) Quantization - The sampled v/g is mapped to the nearest defined discrete v/g level. This step inherently introduces a small amount of rounding error (quantization error)

3) encoding - The quantized v/g level is converted into its corresponding binary number format.



ADC Resolution :- (adhu evalo volts accurate ah erukundra solutthu)

* Resolution defines how finely the ADC can divide the total analog i/p range
* It is expressed in the no. of bits (n)
* the no. of discrete steps available is calculated as 2^n

* A higher resolution means the ADC can detect smaller changes in the analog i/p vtg, resulting in a more precise digital representation.

Ex:

* 8-bit ADC: step : 2^8 = 256 discrete levels

  Digital range : 0 to 255

* 10-bit ADC (used in PIC16F877A)

  step : 2^{10} = 1024 discrete levels

  Digital range : 0 to 1023

* 12-bit ADC

  step : 2^{12} = 4096 " "

  DR : 0 to 4095

Calculating step size (Resolution in volts)

↳ Ithu oru oru step laiyu evalo volts increase ayirukunu solrathu)

step size: the smallest change in analog i/p vtg required to change the digital o/p by one unit (1 LSB - least significant Bit)

Formula:

$$step_size = \frac{V_{ref+} - V_{ref-}}{2^n}$$

This formula is critical for mapping the raw digital num back to a meaningful physical unit in your code

Step size calculation Ex

System parameters: ADC resolution: 10-bit (n=10)

* $V_{ref+} = 5.0V$
* $V_{ref-} = 0.0V$

Calculation:

$step_size = 5.0V - 0.0V / 1024 = 0.00488V$

Conclusion: the digital o/p increases by 1 for every 4.88mV increase in the analog i/p.

ADC calculation: Finding the digit value

$$ADC_Value = \frac{V_{in}}{step\ size}$$

Ex: V_{in} from a sensor is 2.5V

$$ADC_Value = \frac{2.5V}{0.00488V} = 512.29$$

Because the ADC rounds to the nearest whole integer, the result stored in the register will be 512.

i/p vtg		ADC Value
0V	⟶	0
1V	⟶	204
2.5V	⟶	511
5V	⟶	1023

ADCON0 Register

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON
bit-7							bit-0

bit 5-3 CHS2:CHS0: Analog channel select bit

bit 2 GO/DONE: A/D conversion status bit

bit 1 unimplemented: Read as '0'

bit 0 ADCON: A/D ON bit

ADCON1 Register

R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0
ADFM	ADCS2	—	—	PCFG3	PCFG2	PCFG1	PCFG0
bit7							bit0

bit 7 ADFM: A/D Result Format select bit

bit 6: ADCS2: A/D conversion clock select bit (ADCON1 bits in shaded area & in bold)

bit 5-4 unimplemented

bit 3-0 PCFG3:PCFG0: A/D Port configuration control bit

ADCON0 Register

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON
bit-7							bit-0

bit 5-3 CHS2:CHS0: Analog channel select bit

bit 2 GO/DONE: A/D conversion status bit

bit 1 unimplemented: Read as '0'

bit 0 ADCON: A/D ON bit

ADCON1 Register

R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0
ADFM	ADCS2	—	—	PCFG3	PCFG2	PCFG1	PCFG0
bit7							bit0

bit 7 ADFM: A/D Result Format select bit

bit 6 ADCS2: A/D conversion clock select bit (ADCON1 bits in shaded area & in bold)

bit 5-4 unimplemented

bit 3-0 PCFG3:PCFG0: A/D Port configuration control bit

Left vs Right Justification

ADFM = 0
Left
→ No-shifting

SII - 01 1111 1111

H L
01111111 | 11000000

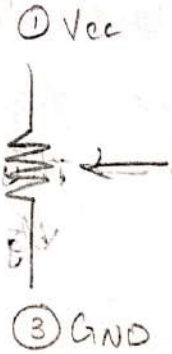
→ very fast
→ LED Brightness motor control
→ ADRESH direct access

ADFM = 1
Right

H L
00000001 / 11111111

→ shifting

HW setup : Interfacing a potentiometer → (POT-HG)



Firmware flow for reading an ADC channel

Step 1 : Configure the specific PORT pin as i/p using the TRIS register

Step 2 : Configure ADCON1 (set Pin as analog, set Vref, select right justification)

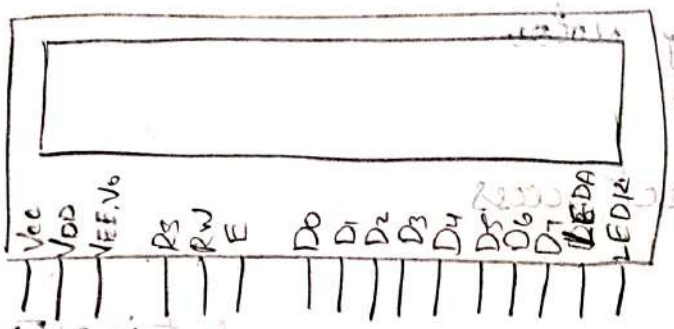
step 3: Configure ADCON0 (select channel, set clock speed, turn ADC ON)

step 4: wait for the Acquisition Time. (The internal holding capacitor needs a few microseconds to charge up to the i/p Vtg)

step 5: set the GO/DONE bit to 1

step 6: poll the " " in a while loop until it reads '0'.

LCD display



1) Pin 1 → Ground → Source pin → This is a ground pin of the LCD → Connected to the ground of the MCU/ PowerSource

2) Pin 2 → Vcc → Source pin → This is the supply Vtg Pin of the LCD → Connected to the supply Pin

3) pin 3 → Vo/VEE → Control Pin → Adjust the contrast of the LCD → Connect to Variable POT that can source 0.5V

4) Pin 4 → Register select → Control pin → Toggles b/w Command/data Register → Connect to MCU Pin & gets either 0 or 1
0 → Command mode
1 → data mode

4) pin 5 → Read/write → Control Pin → Toggle b/w Read/write → 0 → write
1 → Read

5) pin 6 → Enable → Bit → must be held high to perform the Read/write → always held high

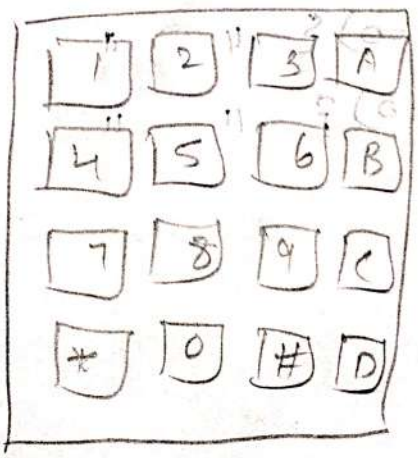
6) pin 7-14 → Data Bit (0-7) → Data/command Pin → Pin are used to send command or data to LCD → In 4-wire only 4 pins (0-3) is connected
In 8-wire All 8 pins (0-7)

7) pin 15 → LED +Ve → LED Pin → normal LED → connect to +5V

pin 16 → LED -Ve → " → " → connect to GND

4X4 matrix Keypad

This 16-button Keypad provides a useful human interface component for microcontroller projects. convenient adhesive backing provides a simple way to mount the keypad in a variety of applications.



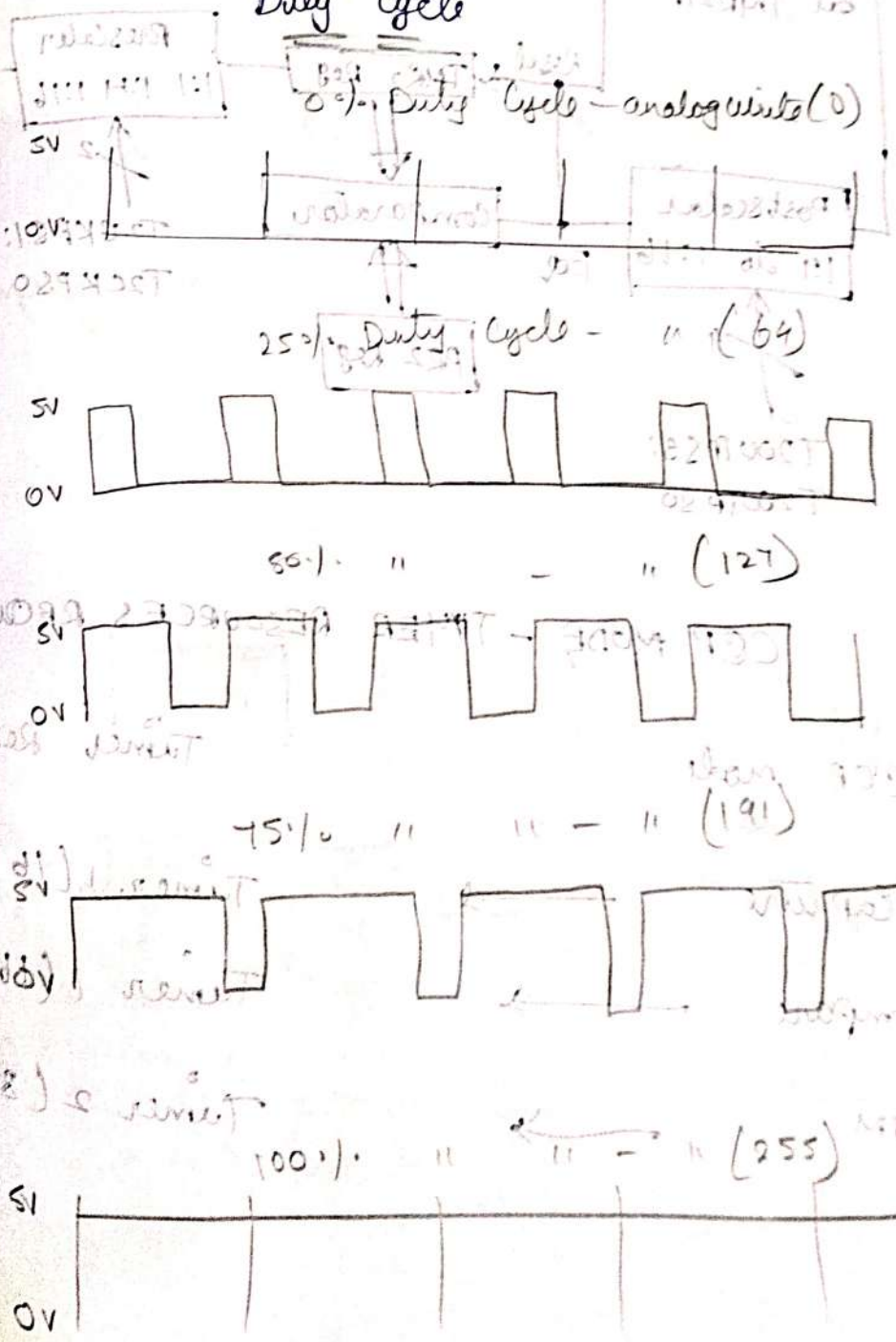
2x) Keypad function (4x3) matrix

```
char Get_Key() {
  while(1) {
    RB0 = 0; RB1 = 1; RB2 = 1; RB3 = 1; // Row 1
    if(RB4==0){__delay_ms(20); while(RB4==0); return '1';}
    if(RB5==0){__delay_ms(20); while(RB5==0); Return '2';}
    if(RB6==0){__delay_ms(20); while(RB6==0); return '3';}
    RB0 = 1; RB1 = 0; RB2 = 1; RB3 = 1; // Row 2
    if(RB4==0){__delay_ms(20); while(RB4==0); Return '4';}
    if(RB5==0){ " " " (RB5==0); " '5';}
    if(RB6==0){ " " " (RB6==0); " '6';}
    RB0 = 1; RB1 = 1; RB2 = 0; RB3 = 1 // Row 3
    if(RB4==0){__delay_ms(20); while(RB4==0); Return '7';}
    if(RB5==0){ " " " (RB5==0); " '8';}
    if(RB6==0){ " " " (RB6==0); " '9';}
    RB0 = 1; RB1 = 1; RB2 = 1; RB3 = 0 // Row 4
    if(RB4==0){__delay_ms(20); while(RB4==0); Return '*';}
    if(RB5==0){ " " " (RB5==0); " '0';}
    if(RB6==0){ " " " (RB6==0); " '#';}
```

Pulse width modulation

It is a technique used to control average power, made eqvl. & create analog results using digital pulses by rapidly switching a signal on and off at a variable duty.

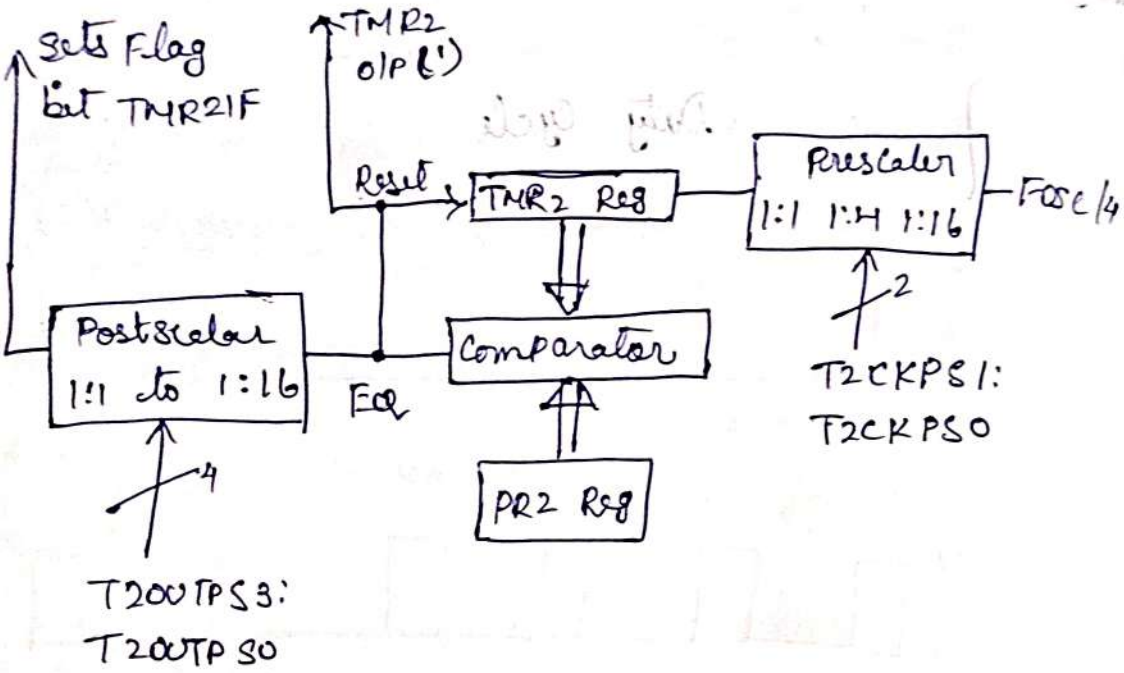
Duty Cycle



1) Timer (1) (TMR1H, TMR1L) (capture, compare)

In Timer 1 we can enable the CCP1 & CCP2

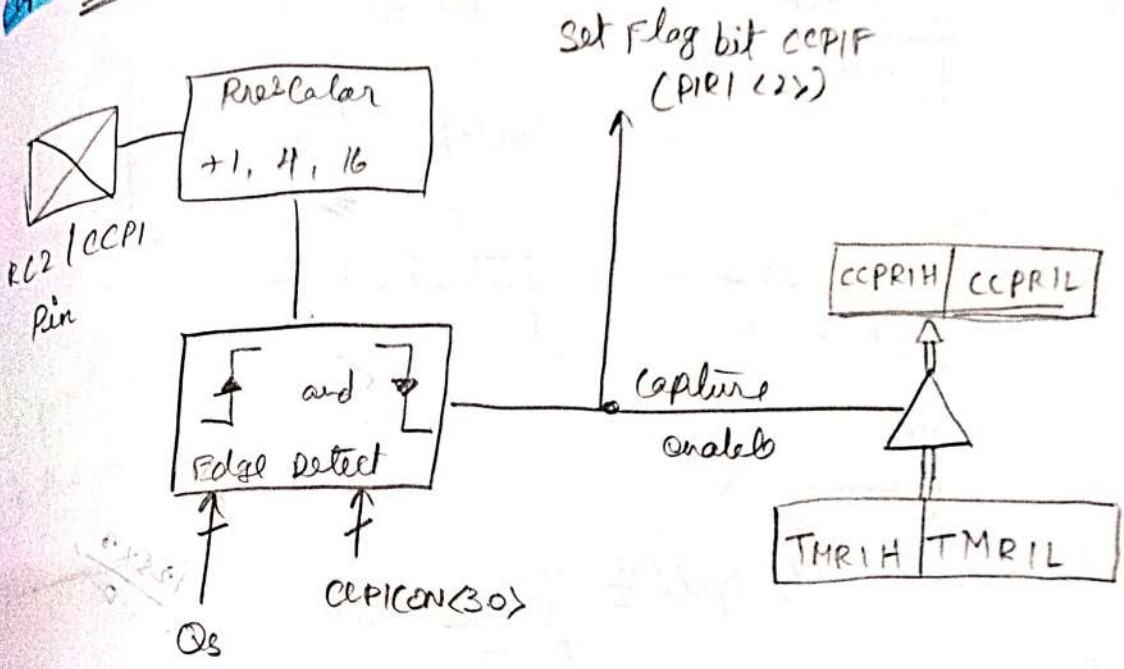
2) Timer (2) (PWM)



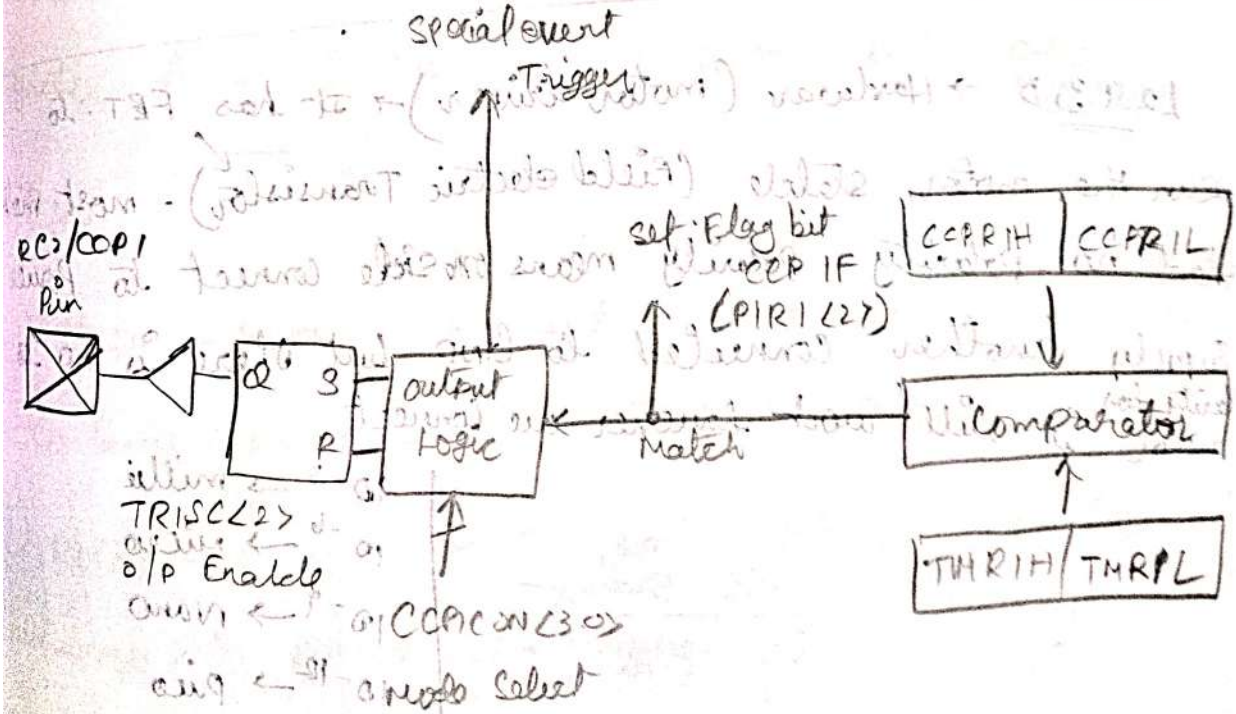
CCP MODE - TIMER RESOURCES REQUIRED

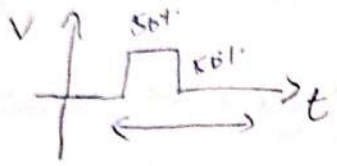
CCP mode	Timer Resource
Capture →	Timer 1 (16 bit)
Compare →	Timer 1 (16 bit)
PWM →	Timer 2 (8 bit)

Capture mode operation Block diagram



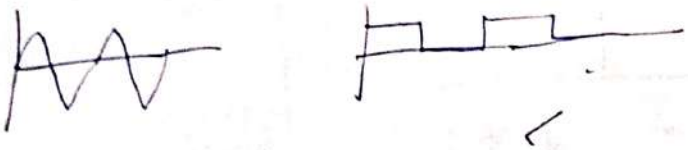
Compare mode operation

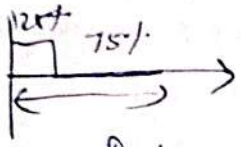




AC (DC)

PMSM BLDC





Duty cycle(%) $\frac{T_{ON}}{T_{ON}+T_{OFF}} \times 100$

$\frac{125 \times 9}{1200}$

$f = \frac{1}{T} = \frac{1}{20}$

$\frac{1}{5 \times 10^3}$ 0.005

L293D → Hardware (motor driver) → It has FET to Run the motor stable (Field electric Transistor) - most motor had no polarity - polarity means one side connect to Power Supply another connected to GND but there is no to Polarity for motor it will work however we connect

Little Endian -

Big Endian -

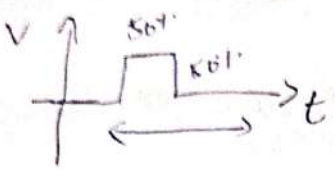
Most Significant Bit

least

1 0 0 1 1 1 0 1

10^{-3} → milli
10^{-6} → micro
10^{-9} → Nano
10^{-12} → pico

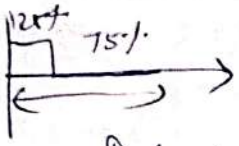
10^{3} → kilo
10^{6} → mega
10^{9} → Giga
10^{12} → tera



AC (DC)

PMSM BLDC





Duty cycle(%) $\frac{T_{ON}}{T_{ON}+T_{OFF}} \times 100$

$\frac{125 \times 9}{1200}$

$f = \frac{1}{T} = \frac{1}{20}$

$\frac{1}{5 \times 10^3}$ 0.005

L293D → Hardware (motor driver) → It has FET to Run the motor stable (Field electric Transistor) - most motor had no polarity - Polarity means one side connect to Power Supply another connected to GND but there is no to Polarity for motor it will work however we connect

Little Endian -

Big Endian -

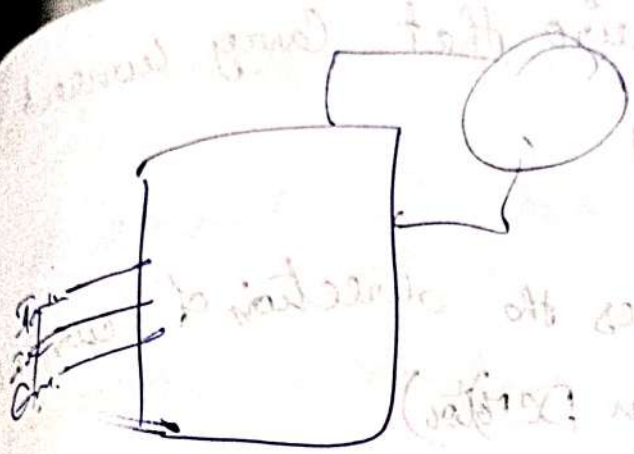
Most Significant Bit

least

1 0 0 1 1 1 0 1

10^{-3} → milli
10^{-6} → micro
10^{-9} → nano
10^{-12} → pico

10^{3} → kilo
10^{6} → mega
10^{9} → giga
10^{12} → tera



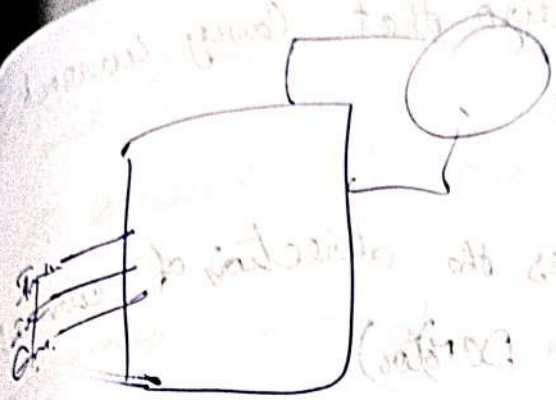
CCP1
CCP2

MOTOR

A motor is an electromechanical device that converts electrical energy into mechanical energy (rotation or linear motion) by interaction b/w a magnetic field & current-carrying conductors.

Parts:

1) Stator → the outer part that remains fixed. It contains winding that produce magnetic field
2) Rotor → inner part that rotates. It has windings or permanent magnets
3) Shaft → Transfers mechanical power from the motor to the load
4) Bearings → Reduce friction b/w rotating parts & support smooth



ccp1
ccp2

MOTOR

A motor is an electromechanical device that converts electrical energy into mechanical energy (rotation or linear motion) by interaction b/w a magnetic field & current-carrying conductors.

Parts:

1) Stator → the outer part that remains fixed. It contains winding that produce magnetic field
2) Rotor → inner part that rotates. It has windings or permanent magnets
3) Shaft → Transfers mechanical power from the motor to the load
4) Bearings → Reduce friction b/w rotating part & supports smoth rotation

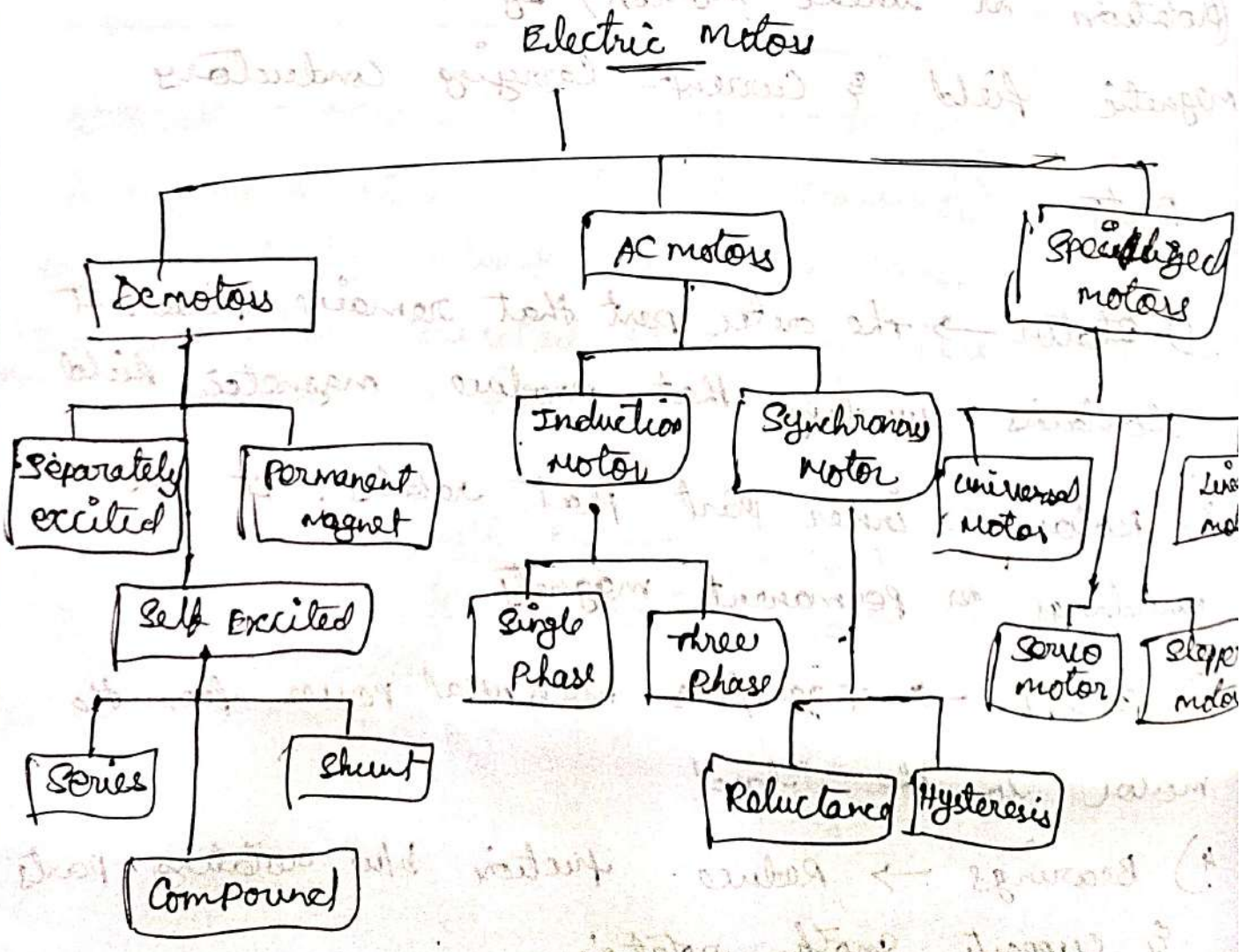
5) windings → coils of wire that carry current & produce magnetic field

6) commutator → reverses the direction of current in armature coils (used in DC motor)

7) Brushes → provides electrical contact b/w the commutator & external circuit (used in DC motors)

8) cooling Fan → helps in heat dissipation & keeps the motor cool

9) End shield / Frame → Holds the parts together & protects internal components



Types of motors

It specified into two categories DC & AC motor

1) DC motor → operating using direct current supply

a) Brushed DC motor → for Toys, small, Pumps, electric drills, wipers

b) Brushless DC " → Drones, EVs, Cooling fans, Robotics

c) series DC " → Cranes, electric trains, hoists

d) shunt " " → Lathes, conveyors, fans, blowers

e) compound " " → Elevators, Press machines, Rolling mills

2) AC motor → operating using AC supply

a) single-phase Induction motor → ceiling fans, washing machines, water pumps, air coolers.

b) three-phase " " → Industrial purpose, Compressors

c) synchronous " → CNC machines, Power plants, Industrial drives, Precision equipment

3) special purpose motors

a) stepper motor → 3D Printer, CNC machine, robotics Camera positioning

b) servo motor → Robotic arms, CNC machines, RC vehicles automation.

c) universal motor → mixer, vaccum cleaner, hair dryer, power tools

D) linear " → Automatic doors, maglev trains, industrial automation

E) geared motor → Robots, conveyor belts, automated gates

Duty cycle

It is the percentage of time a signal is HIGH (ON) in one complete period

Formula

$$\text{Duty cycle } D = \frac{T_{ON}}{T_{TOTAL}\ (ON+OFF)} \times 100\%$$

Capture

Ex:
```
#include <xc.h>
Void main()
{
 unsigned int value;
 T1CON = 0x01;
 CCP1CON = 0x05;
 while (1)
 {
  while (!CCP1IF);
    CCP1IF = 0
```

c) universal motor → mixer, vaccum cleaner, hair dryer, power tools

D) linear " → Automatic doors, maglev trains, industrial automation

E) geared motor → Robots, conveyor belts, automated gates

Duty cycle

It is the percentage of time a signal is HIGH (ON) in one complete period

Formula

$$\text{Duty cycle } D = \frac{T_{ON}}{T_{TOTAL}\ (ON+OFF)} \times 100\%$$

Capture

Ex:
```
#include <xc.h>
Void main()
{
 unsigned int value;
 T1CON = 0x01;
 CCP1CON = 0x05;
 while (1)
 {
  while (!(CCP1IF));
    CCP1IF = 0
```

value = ((CCPR1H<<8) | CCPR1L);

}

}

Compare

Ex: #include <xc.h>

void main()

{

TRISC0 = 0;

T1CON = 0x01;

CCPR1 = 5000;

CCP1CON = 0x09;

while (1);

}

PWM

Ex: #include <xc.h>

#define _XTAL_FREQ xxxxxxx

void PWM_Init()

{

TRISC3 = 0;

PR2 = 249;

CCP1CON = 0x0C;

CCPR1L = 125;

T2CON = 0x04;

}

void main()

{

PWM_Init();

while(1)

{

}

}

PR2 Calculation

$$PR2 = \frac{Fosc}{4 \times Prescaler \times F_{PWM}} - 1$$

$$= \frac{20,000,000}{4 \times 4 \times 5000} - 1$$

$$= \frac{20,000,000}{80,000} - 1 \Rightarrow 250 - 1$$

$$= 249$$

value = ((CCPR1H<<8) | CCPR1L);

}
}

Compare

Ex: #include <xc.h>

void main()

{

TRISC2 = 0;

T1CON = 0x01;

CCPR1 = 5000;

CCP1CON = 0x09;

while (1);

}

PWM

Ex: #include <xc.h>

#define _XTAL_FREQ 20000000

void PWM_Init()

{

TRISC3 = 0;

PR2 = 249;

CCP1CON = 0x0C;

CCPR1L = 125;

T2CON = 0x04;

}

void main()

{

PWM_Init();

while(1)

{

}

}

PR2 Calculation

$$PR2 = \frac{Fosc}{4 \times Prescaler \times F_{PWM}} - 1$$

$$= \frac{20,000,000}{4 \times 4 \times 5000} - 1$$

$$= \frac{20,000,000}{80,000} - 1 \Rightarrow 250 - 1$$

$$= 249$$

Communication Protocol

* It is the exchange of data b/w devices
* Enables interaction b/w sensors, microcontroller & computers

Ex:

computer to printer
Micron - sensor
mobile phone - Internet

Importance

* A comm protocol is a set of rules governing data exchange.
* Defines how data is transmitted & received
* Ensures accurate & reliable comm b/w devices

Key functions

Data Formatting
Synchronization
Error detection
Transmission speed Control

Types

1) serial comm
   ↳ Data sent bit by bit
   ↳ uses fewer wires
2) parallel comm

↳ data sent multiple bits at once

↳ faster but complex

al Commu types

ynchronous

↳ uses clock signal

↳ SPI, I2C

Asynchronous

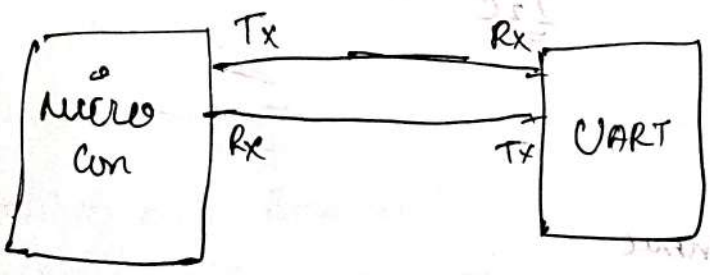
↳ no clock sgnl

↳ UART, CAN

UART :-

features :

* Asynchronous commu
* no clock sgnl required
* uses Tx & Rx lines
* simple & cost-effective
* widely used for debugging



Start D0 D1 D2 D3 D4 D5 D6 D7 parity stop

↳ Data sent multiple bits at once
↳ faster but complex

ial Commu types

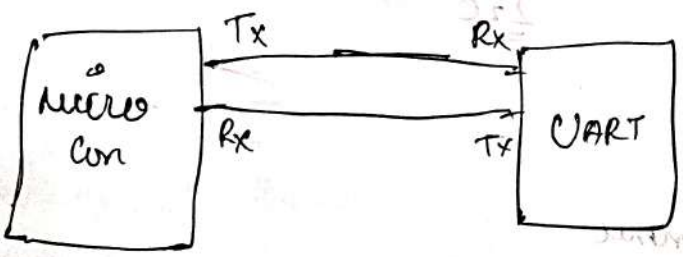
synchronous
↳ uses clock signal
↳ SPI, I2C

Asynchronous
↳ no clock sgnl
↳ UART, CAN

~~types~~ UART :-

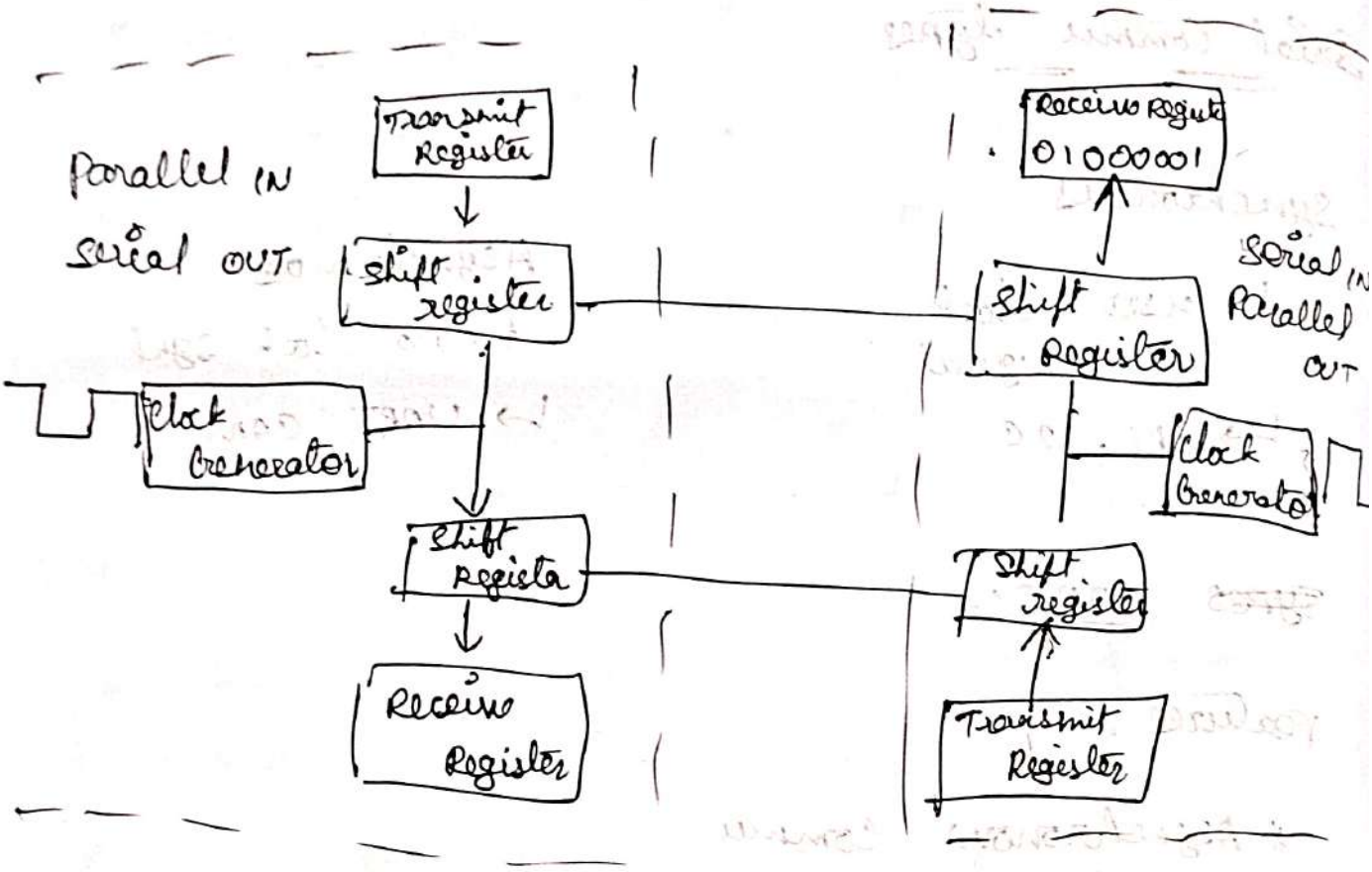
features ;

* Asynchronous Commu
* no clock sgnl required
* uses Tx & Rx lines
* simple & cost-effective
* widely used for debugging



Start D0 D1 D2 D3 D4 D5 D6 D7 parity stop

UART PERIPHERALS



UART Application

* Printer
* Gsm
* Bluetooth

I2C

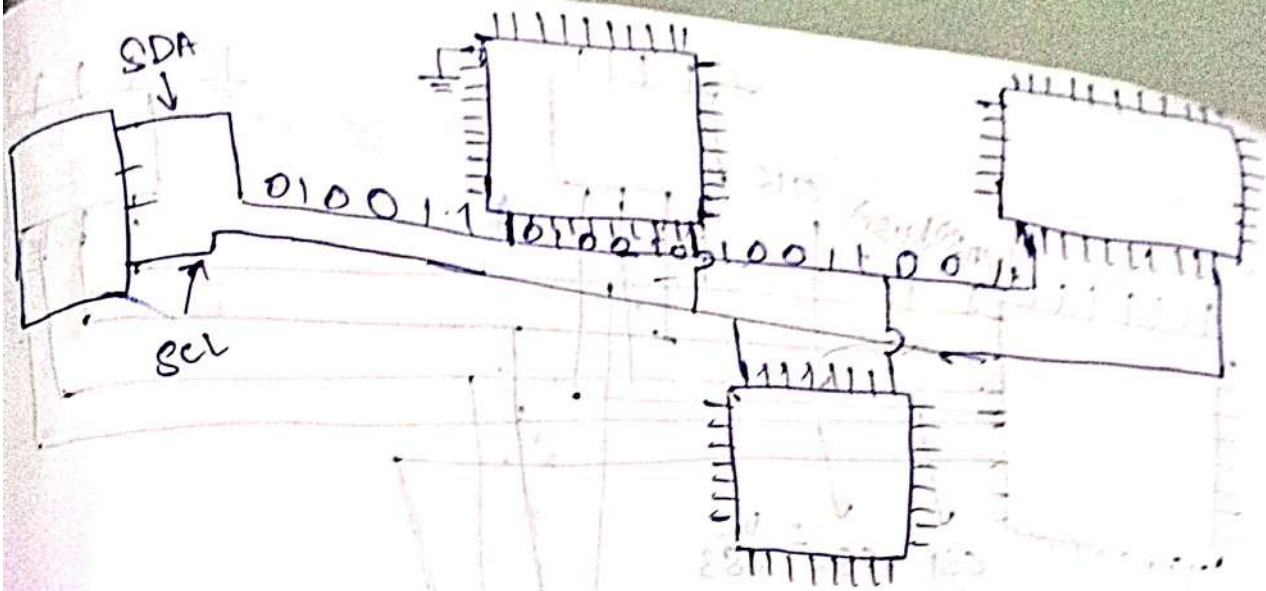


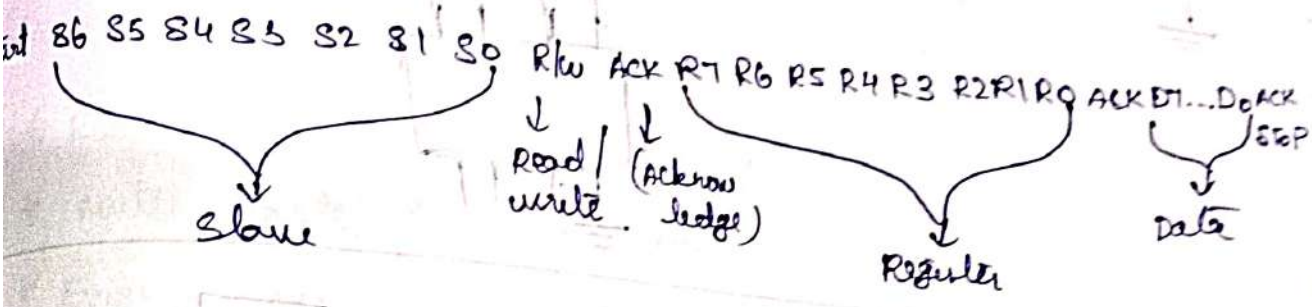
Features

* Two - wire Communication
* Supports multiple masters & Slaves
* Uses addressing system

Lines used

* SDA - Serial Data
* SCL - Serial clock





Modes of I2C

Standard - Upto 100 KbPS
Fast - Upto 400 KbPS
Fast + - upto 1 MbPS
High speed - upto 3.4 MbPS

Application

* EEPROM
* RTC modules
* Sensors

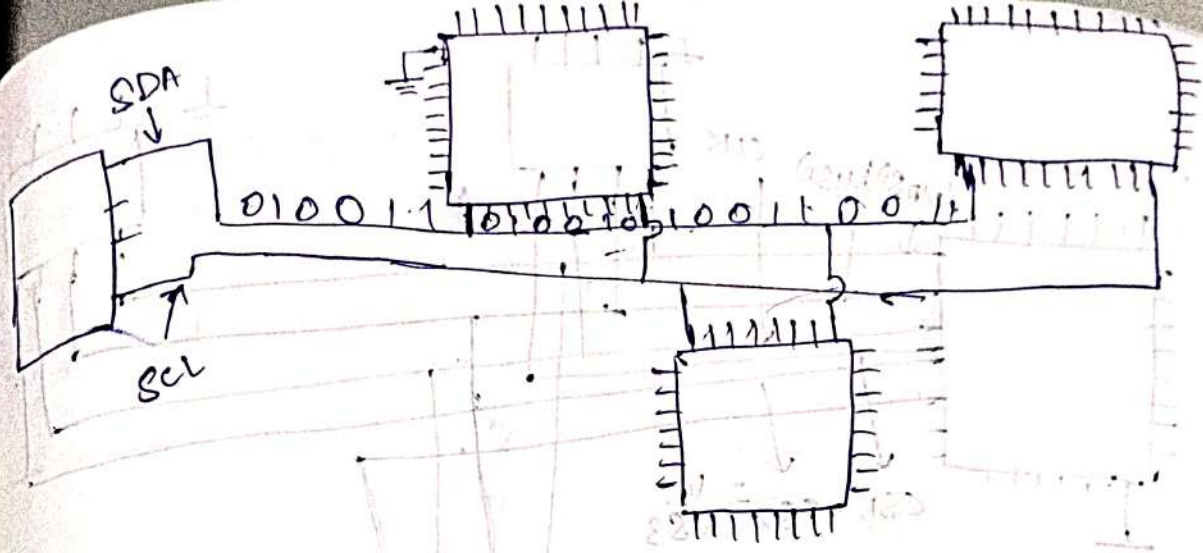
SPI

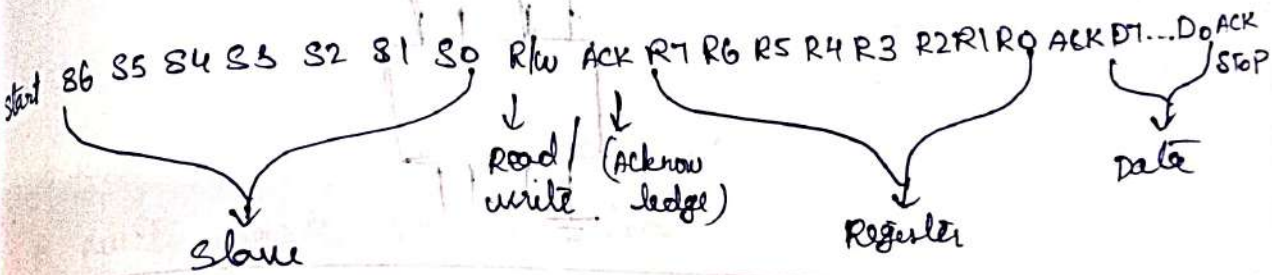
Features

* Synchronous Commn
* Full - duplex data transfer
* High - Speed Protocol

Lines used:

* MOSI - master out Salve In
* MISO - " In " out
* SCLK - Serial Clock
* SS - Slave select





Modes of I2C

Standard - upto 100 KbPS
Fast - upto 400 KbPS
Fast + - upto 1 MbPS
High speed - upto 3.4 MbPS

Application

* EEPROM
* RTC modules
* sensors

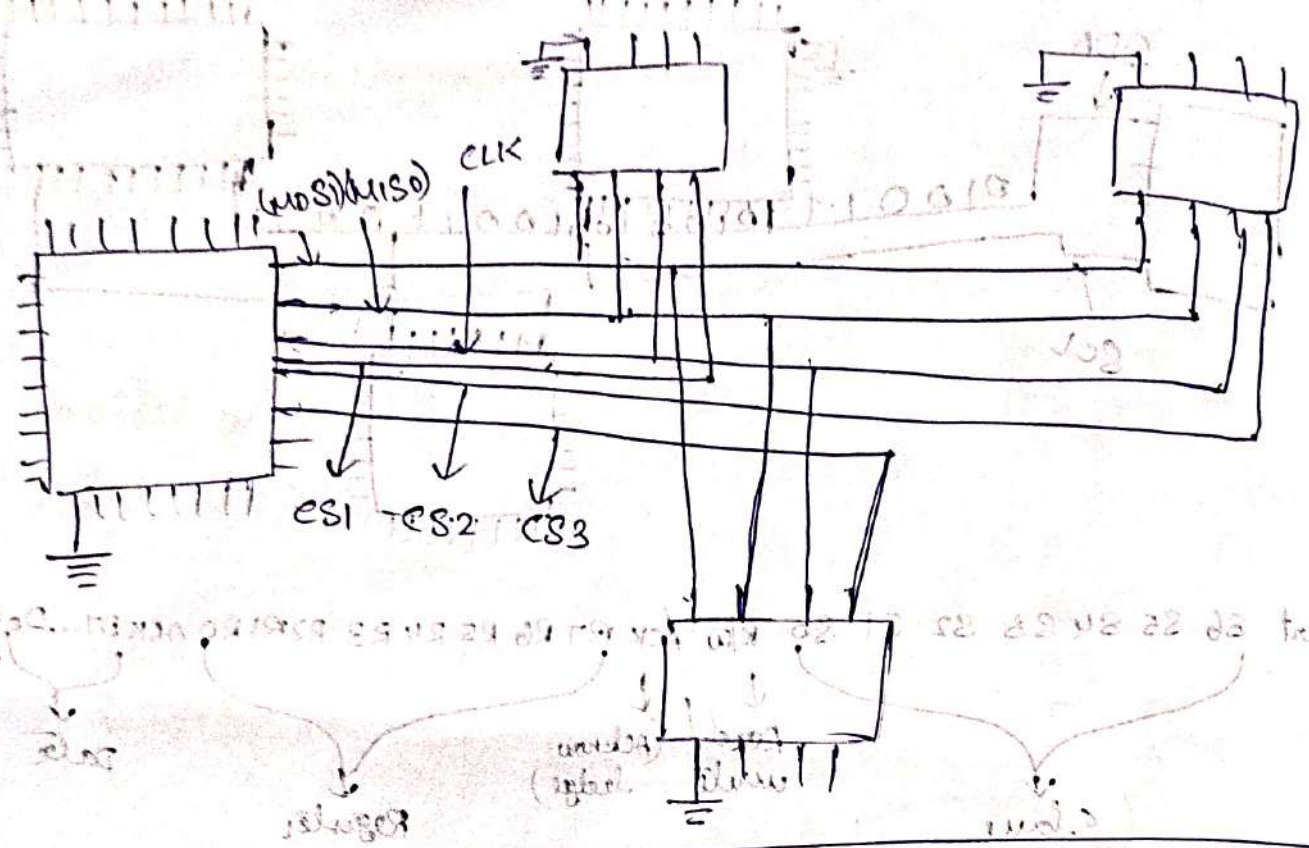
SPI

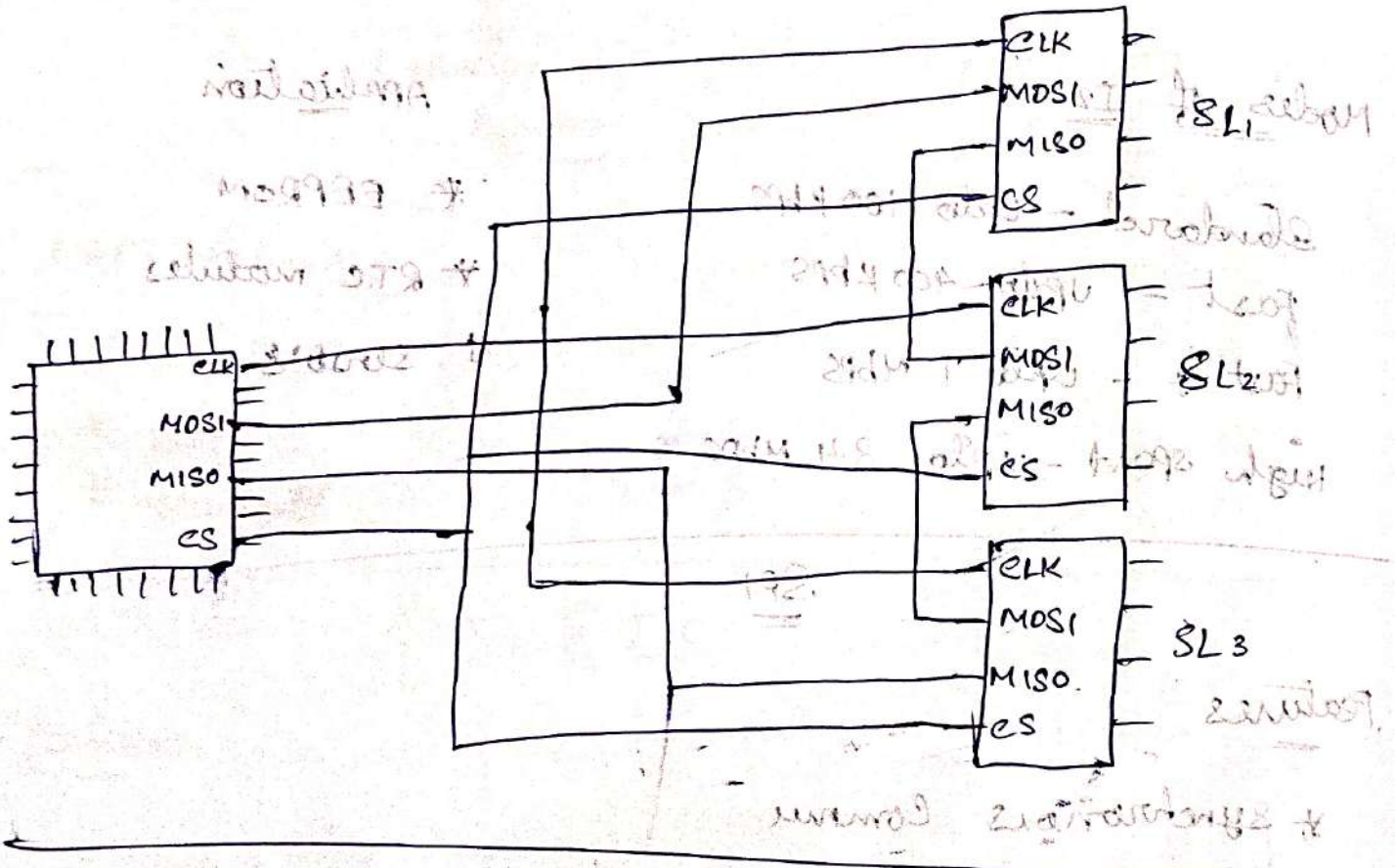
Features

* Synchronous Comm
* Full - duplex data transfer
* High - Speed Protocol

lines used:

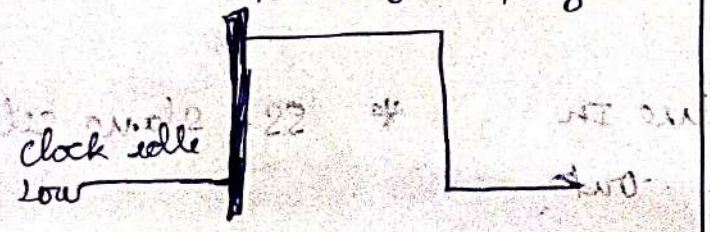
* MOSI - master out Salve In
* MISO - " In " out
* SCLK - Serial Clock
* SS - slave select



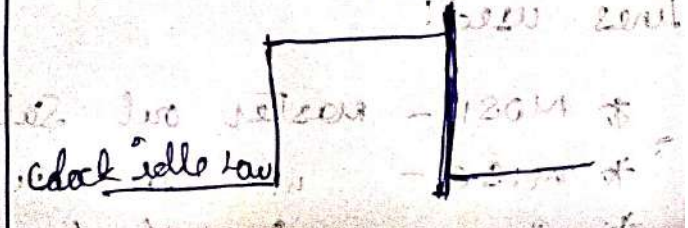


Application : SD Cards, ADC / DAC modules, Display

Mode 0

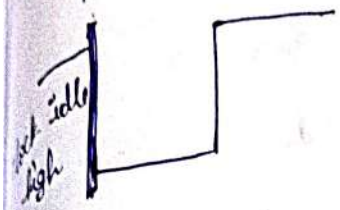


Mode 1



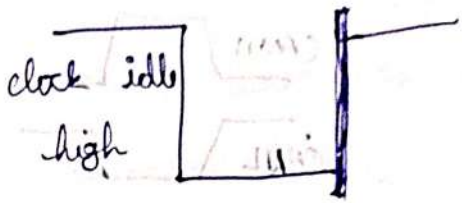
node 2

1st edge sampling



node 3

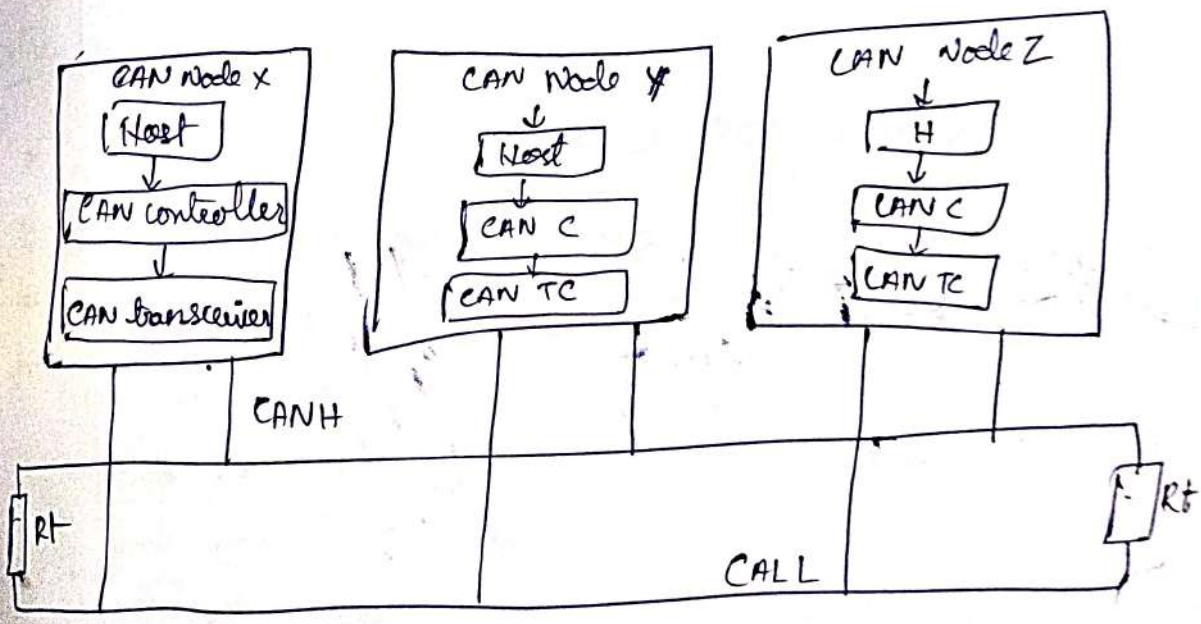
2nd edge sampling



CAN (controller area n/w)

features :-

* Robust & reliable Protocol
* multi-master communication
* Error detection & fault tolerance



SOF	Msg identifier	control	Data	CRC	ACK	EF
1 bit	11 or 29 bit	6 bit	upto 64 bit	16 bit	2 bit	7 bit

Application :-

* Automation * Industrial automation * Robotics

1 → Recessive , 0 → Dominant

